

数据结构

第三章 栈和队列

栈

栈的应用举例

队列

前言

□ 栈和队列是两种特殊的线性表，其特殊性在于栈和队列的基本操作是线性表的子集，它们是**操作受限**的线性表，可称为限定性的DS.

◆ 硬堆栈：利用CPU中的某些寄存器组或类似的硬件或使用内存的特殊区域来实现。这类堆栈容量有限，但速度很快；

◆ 软堆栈：这类堆栈主要在内存中实现。堆栈容量可以达到很大。在实现方式上，又有动态方式和静态方式两种。

本章将讨论栈和队列的基本概念、存储结构、基本操作以及这些操作的具体实现。

第一节 栈

□ 栈(stack)

■ 限定仅在表尾进行插入或删除操作的线性表

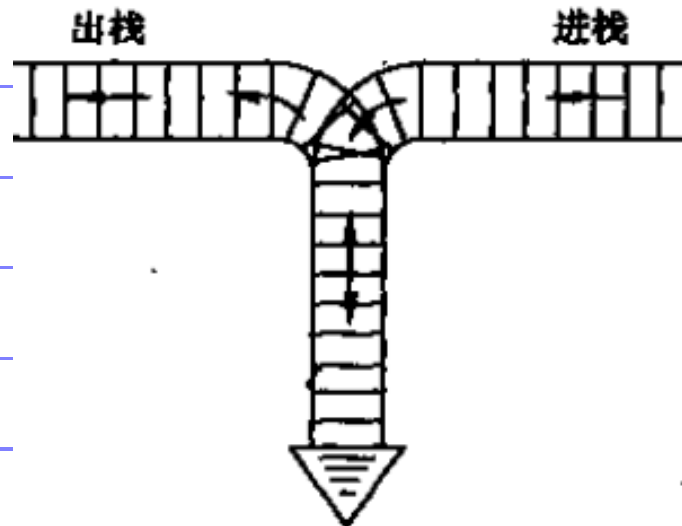
◇ 表尾—**栈顶**—允许进行插入、删除操作的一端, 用栈顶指针(**top**)来指示栈顶元素.

◇ 表头—**栈底**—是固定端

◇ 不含元素的空表称**空栈**

(Last In First Out)

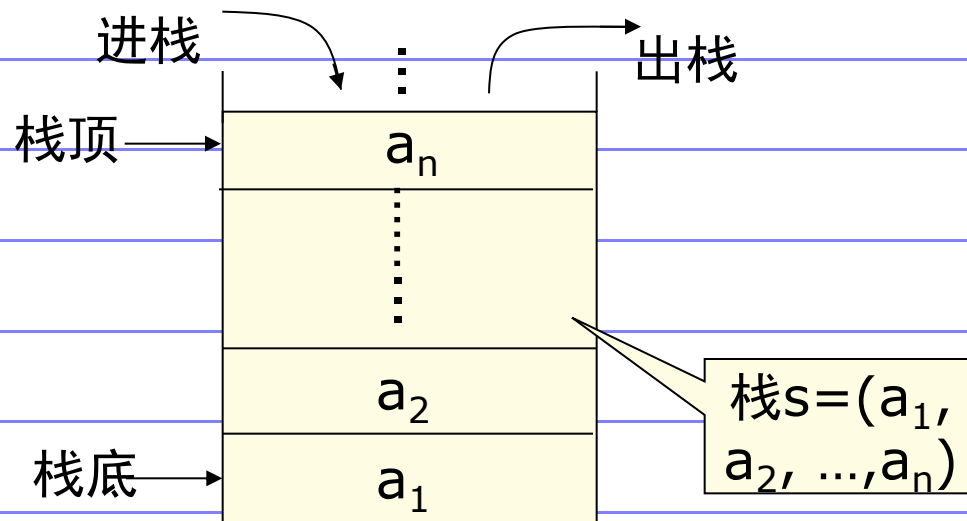
■ 特点: 先进后出 (**FILO**) 或后进先出 (**LIFO**)



第一节 栈

设栈 $S=(a_1, a_2, \dots, a_n)$ ，则 a_1 称为栈底元素， a_n 为栈顶元素，如下图所示。

栈中元素按 $a_1, a_2, \dots, a_n$ 的次序进栈，退栈的第一个元素应为栈顶元素。即栈的修改是按后进先出的原则进行的。



一个栈的进栈序列是 a, b, c, d, e ，则不可能的输出序列是_____。

- A. edcba B. decba C. dceab D. abcde .

栈的抽象数据类型定义

ADT Stack{

- 数据对象: $D = \{ a_i \mid a_i \in \text{ElemSet}, i=1,2,\dots,n, n \geq 0 \}$
- 数据关系: $R = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2,3,\dots,n \}$
- 基本操作: 初始化、进栈、出栈、取栈顶元素等

} ADT Stack

栈的基本操作

□ 教材P44-45关于栈的抽象描述

InitStack(&S)

操作结果:构造一个空栈 S。

DestroyStack(&S)

初始条件:栈 S 已存在。

操作结果:栈 S 被销毁。

ClearStack(&S)

初始条件:栈 S 已存在。

操作结果:将 S 清为空栈。

StackEmpty(S)

初始条件:栈 S 已存在。

操作结果:若栈 S 为空栈,
则返回 TRUE, 否则 FALSE。

StackLength(S)

初始条件:栈 S 已存在。

操作结果:返回 S 的元素个数, 即栈的长度。

GetTop(S, &e)

初始条件:栈 S 已存在且非空。

操作结果:用 e 返回 S 的栈顶元素。

Push(&S, e)

初始条件:栈 S 已存在。

操作结果:插入元素 e 为新的栈顶元素。

Pop(&S, &e)

初始条件:栈 S 已存在且非空。

操作结果:删除 S 的栈顶元素, 并用 e
返回其值。

StackTraverse(S, visit())

初始条件:栈 S 已存在且非空。

操作结果:从栈底到栈顶依次对 S 的每个数据元素调用函数 visit()。
一旦 visit() 失败, 则操作失效。

栈的顺序存储结构

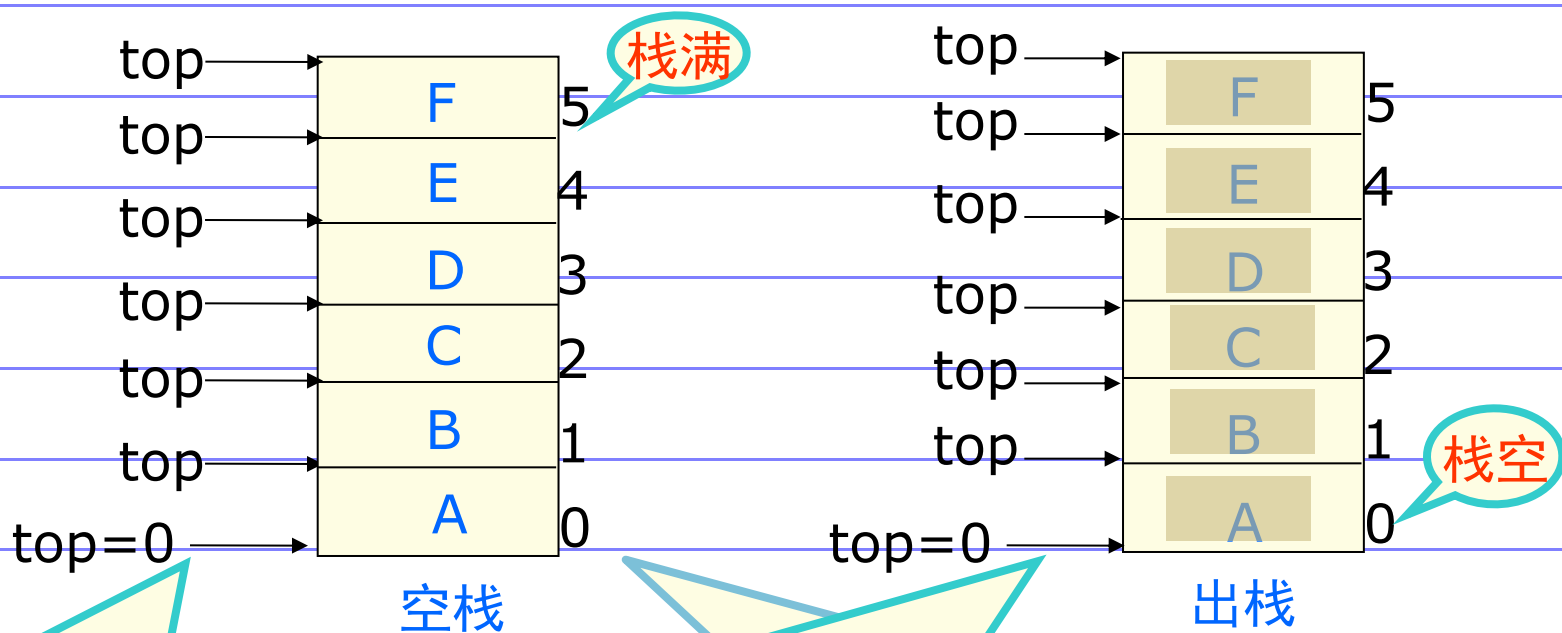
栈的顺序存储结构简称为顺序栈，和线性表相类似，用一维数组来存储栈。根据数组是否可以根据需要增大，又可分为静态顺序栈和动态顺序栈。

- ◆ 动态顺序栈可以根据需要增大栈的存储空间，但实现稍为复杂。

- ◆ 静态顺序栈实现简单，但不能根据需要增大栈的存储空间；

栈的顺序存储结构

- ❑ 顺序栈—实现：一维数组s[M]
- ❑ 当堆栈中数据全部弹出后，在内存中的是什么？



栈顶指针top, 指向实际栈顶后的空位置, 初值为0

top=0 栈空, 此时出栈则下溢 (underflow)
top=M 栈满, 此时入栈则上溢 (overflow)

栈的静态顺序存储表示

采用静态一维数组来存储栈。

栈底固定不变的，而栈顶则随着进栈和退栈操作变化的，

- ◆ 栈底固定不变的；栈顶则随着进栈和退栈操作而变化，用一个整型变量 top （称为栈顶指针）来指示实际栈顶后的空位置。
- ◆ 用 $\text{top}=0$ 表示栈空的初始状态
- ◆ 结点进栈：首先将数据元素保存到栈顶(top 所指的当前位置)，然后执行 top 加1，使 top 指向新的空位置。

栈的静态顺序存储表示

◆ **结点出栈**：首先执行**top**减1，使**top**指向栈顶位置，然后把**top**指向的栈顶元素取出。

若栈的数组有**Maxsize**个元素，则**top=Maxsize**时栈满。下图是一个大小为4的栈的变化示意图。

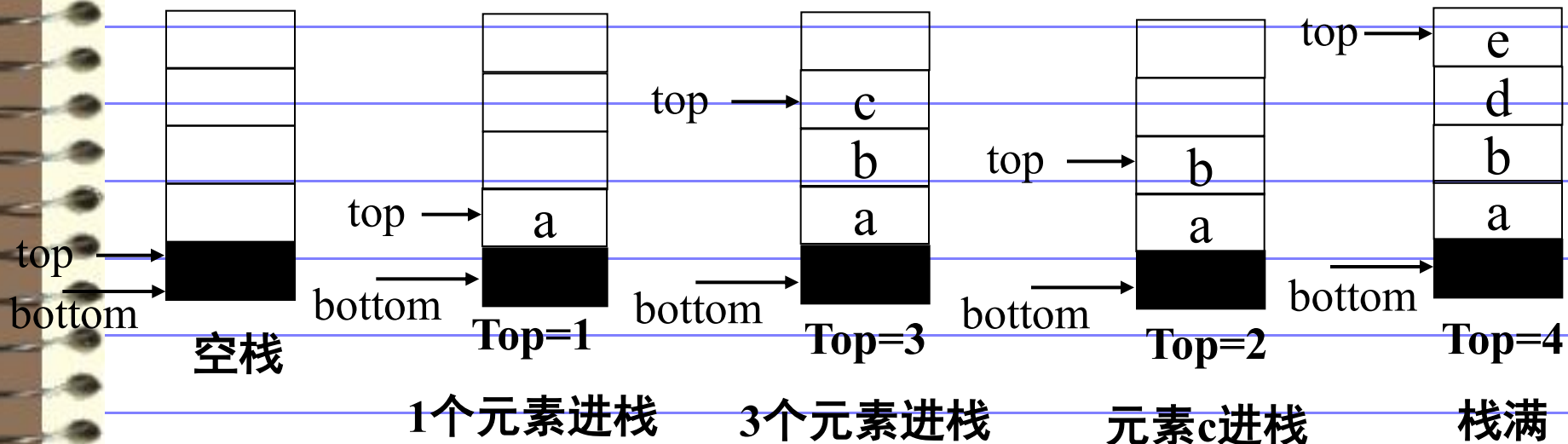


图3-3 静态堆栈变化示意图

栈的静态顺序存储表示--栈的类型定义

```
# define MAX_STACK_SIZE 100 // 栈向量大小
```

```
# typedef int ElemType ;
```

```
typedef struct sqstack
```

```
{ ElemType  stack_array[MAX_STACK_SIZE] ;
```

```
    int top;
```

```
}SqStack ;
```

栈的静态顺序存储表示-栈的初始化

```
SqStack Init_Stack(void)
```

```
{    SqStack S ;
```

```
    S.top=0 ; return(S) ;
```

```
}
```

栈的静态顺序存储表示--压栈(元素进栈)

```
Status push(SqStack S , ElemType e)  
    /* 使数据元素e进栈成为新的栈顶 */  
    { if (S.top==MAX_STACK_SIZE)  
        return ERROR;    /* 栈满, 返回错误标  
        志 */  
        S.stack_array[S.top]=e ; /* e成为新  
        的栈顶 */  
        S.top++ ;  
        return OK;    /* 压栈成功 */  
    }
```

栈的静态顺序存储表示--弹栈(元素出栈)

```
Status pop( SqStack S, ElemType *e )  
    /*弹出栈顶元素*/  
    { if ( S.top==0 )  
        return ERROR ; // 栈空, 返回错误标志  
    S.top-- ;  
    *e=S.stack_array[S.top] ;  
    return OK ;  
}
```

当栈满时做进栈运算必定产生空间溢出, 简称“上溢”。上溢是一种出错状态, 应设法避免。

当栈空时做退栈运算也将产生溢出, 简称“下溢”。下溢则可能是正常现象, 因为栈在使用时, 其初态或终态都是空栈, 所以下溢常用来作为控制转移的条件。

栈的动态顺序存储表示

采用动态一维数组来存储栈。所谓动态，指的是栈的大小可以根据需要增加。

◆ 用**bottom**表示栈底指针，栈底固定不变的；栈顶则随着进栈和退栈操作而变化。用**top**（称为栈顶指针）指示当前栈顶位置。

◆ 用**top=bottom**作为栈空的标记，每次**top**指向栈顶数组中的下一个存储位置。

◆ **结点进栈**：首先将数据元素保存到栈顶(**top**所指的当前位置)，然后执行**top**加1，使**top**指向栈顶的下一个存储位置；

栈的动态顺序存储表示

◆ **结点出栈**：首先执行**top**减1，使**top**指向栈顶元素的存储位置，然后将栈顶元素取出。图3-2是一个动态栈的变化示意图。

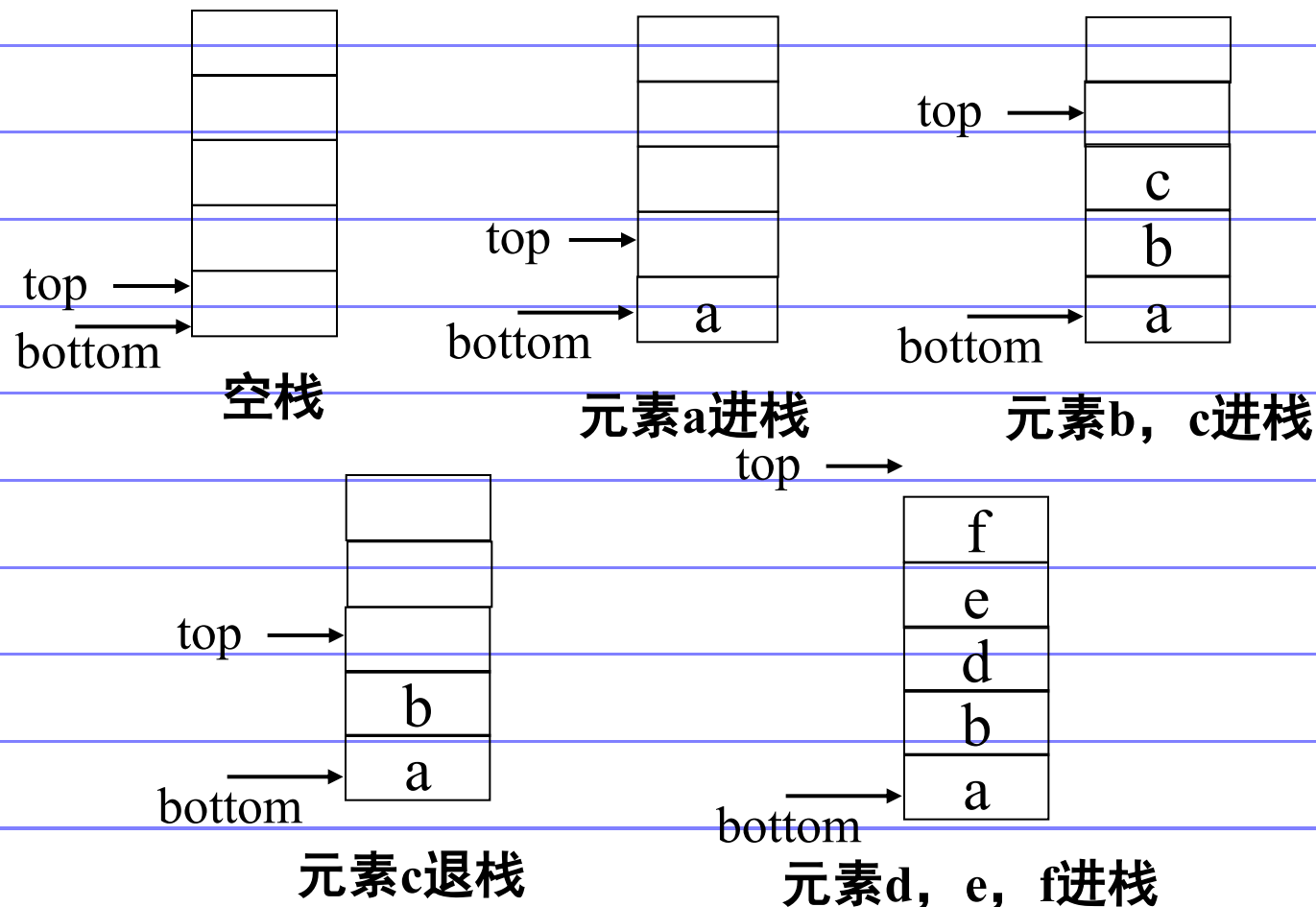


图3-2 (动态)堆栈变化示意图

栈的顺序存储表示

```
#define STACK_INIT_SIZE 100    栈初始向量大小
#define STACKINCREMENT 10      存储空间分配增量

typedef struct {
    SElemType *base; //在栈构造之前和销毁之后，base值为NULL
    SElemType *top;  // 栈顶指针
    int stacksize;    // 当前已分配空间，以元素为单位
} SqStack;
```

栈的基本操作

- ❑ Status InitStack(SqStack &S);
- ❑ Status DestroyStack(SqStack &S);
- ❑ Status ClearStack(SqStack &S);
- ❑ Status StackEmpty(SqStack S);
- ❑ int StackLength(SqStack S);
- ❑ Status GetTop(SqStack S, SElemtype &e);
- ❑ Status Push(SqStack &S, SElemType e);
- ❑ Status Pop(SqStack &S, SElemType &e);
- ❑ Status StackTraverse(SqStack S, Status(*visit)());

顺序栈算法---栈的初始化

```
Status InitStack(SqStack &S) {  
    S.base =  
        (SElemType*)malloc(STACK_INIT_SIZE*sizeof(SElemType));  
    if(!S.base) exit(OVERFLOW);  
    S.top = S.base;  
    S.stacksize = STACK_INIT_SIZE;  
    return OK;  
}
```

顺序栈算法

□ 入栈算法、出栈算法

```
Status push(SqStack &S, SElemType e) {  
    if(S.top - S.base >= S.stacksize) {  
        S.base = (SElemType *)realloc(S.base,  
            (S.stacksize+STACKINCREMENT)*sizeof(SElemType));  
        if(!S.base) exit(OVERFLOW);  
        S.top = S.base + S.stacksize;  
        S.stacksize += STACKINCREMENT;  
    }  
    *S.top++ = e;  
    return OK;  
}
```

顺序栈算法

```
Status Pop(SqStack &S, SElemType &e) {  
    if(S.top==S.base) return ERROR;  
    e = *--S.top;  
    return OK;  
}
```

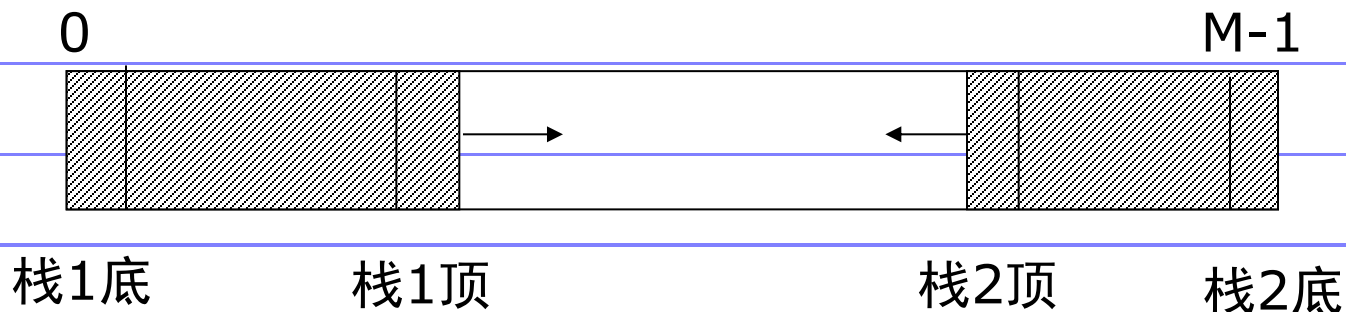
弹栈(元素出栈)

```
Status GetTop(SqStack S, SElemType &e) {  
    if(S.top==S.base) return ERROR;  
    e = *(S.top-1);  
    return OK;  
}
```

- 用数组实现的顺序栈的评价
 - 栈的容量在使用前难以估计
 - 操作简便

共享顺序栈

- 顺序栈所需容量在使用前难以估计，而且有些CPU内存有限，可供堆栈使用的空间有限，如单片机。常在一个程序中将两个堆栈使用的空间放在一起。



栈的链式存储结构

栈的链式存储结构称为链栈，是运算受限的单链表。其插入和删除操作只能在表头位置上进行。因此，链栈没有必要像单链表那样附加头结点，栈顶指针 top 就是链表的头指针。

链栈的结点类型说明如下：

```
typedef struct Stack_Node
{ ElemType data;
  struct Stack_Node *next;
} Stack_Node;
```

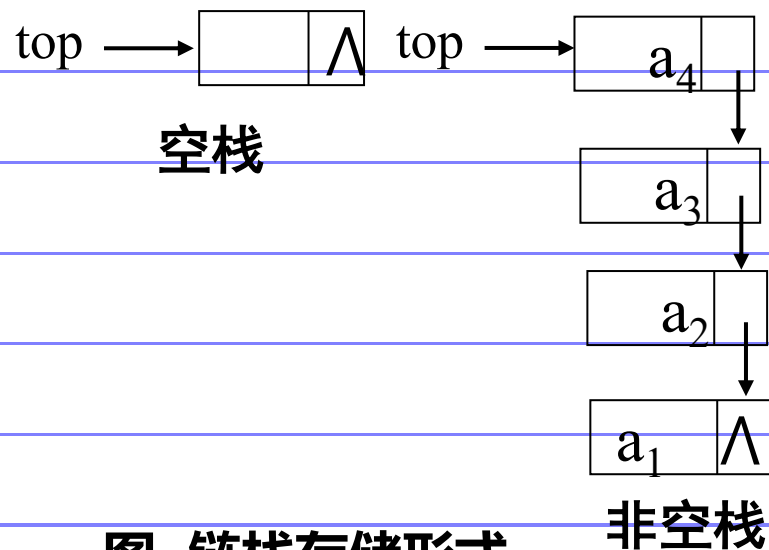


图 链栈存储形式

链栈算法

□ 链栈初始化

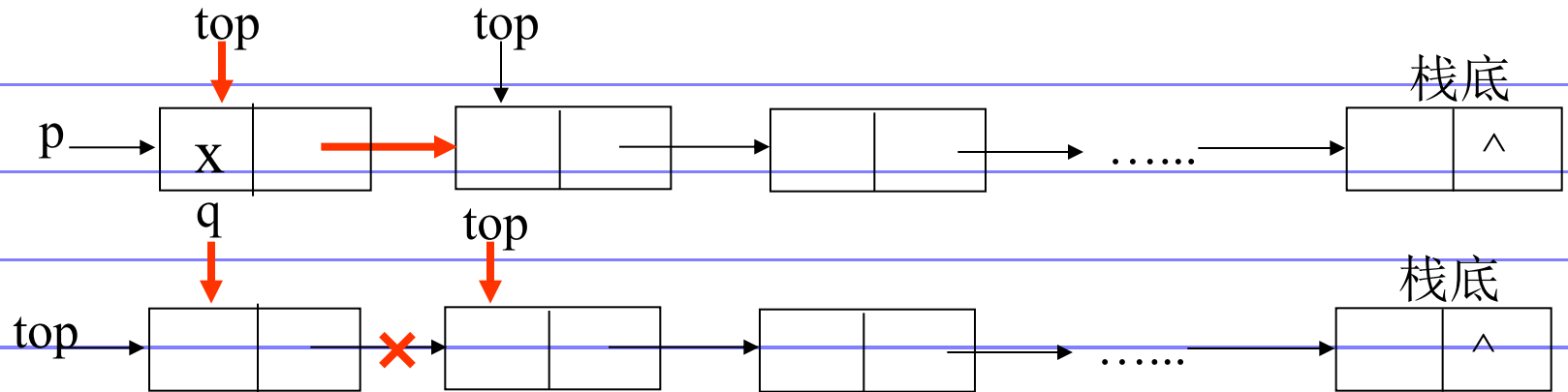
```
Stack_Node *Init_Link_Stack(void)
{
    Stack_Node *top ;
    top = NULL ;
    return(top) ;
}
```

链栈算法

❑ 链栈通常都在链表头端操作。在尾端操作呢？

```
Stack_Node *LSPush(  
    Stack_Node *top, int x)  
{  
    Stack_Node *p;  
    p = (Stack_Node *)  
        malloc(sizeof(Stack_Node));  
    p->data = x;  
    p->next = top;  
    top = p;  
    return(p);  
}
```

```
Stack_Node *LSPop(  
    Stack_Node *top, int *x)  
{  
    Stack_Node *q;  
    if (top) {  
        q = top;  
        *x = top->data;  
        top = top->next;  
        free(q);  
    }  
    return(top);  
}
```



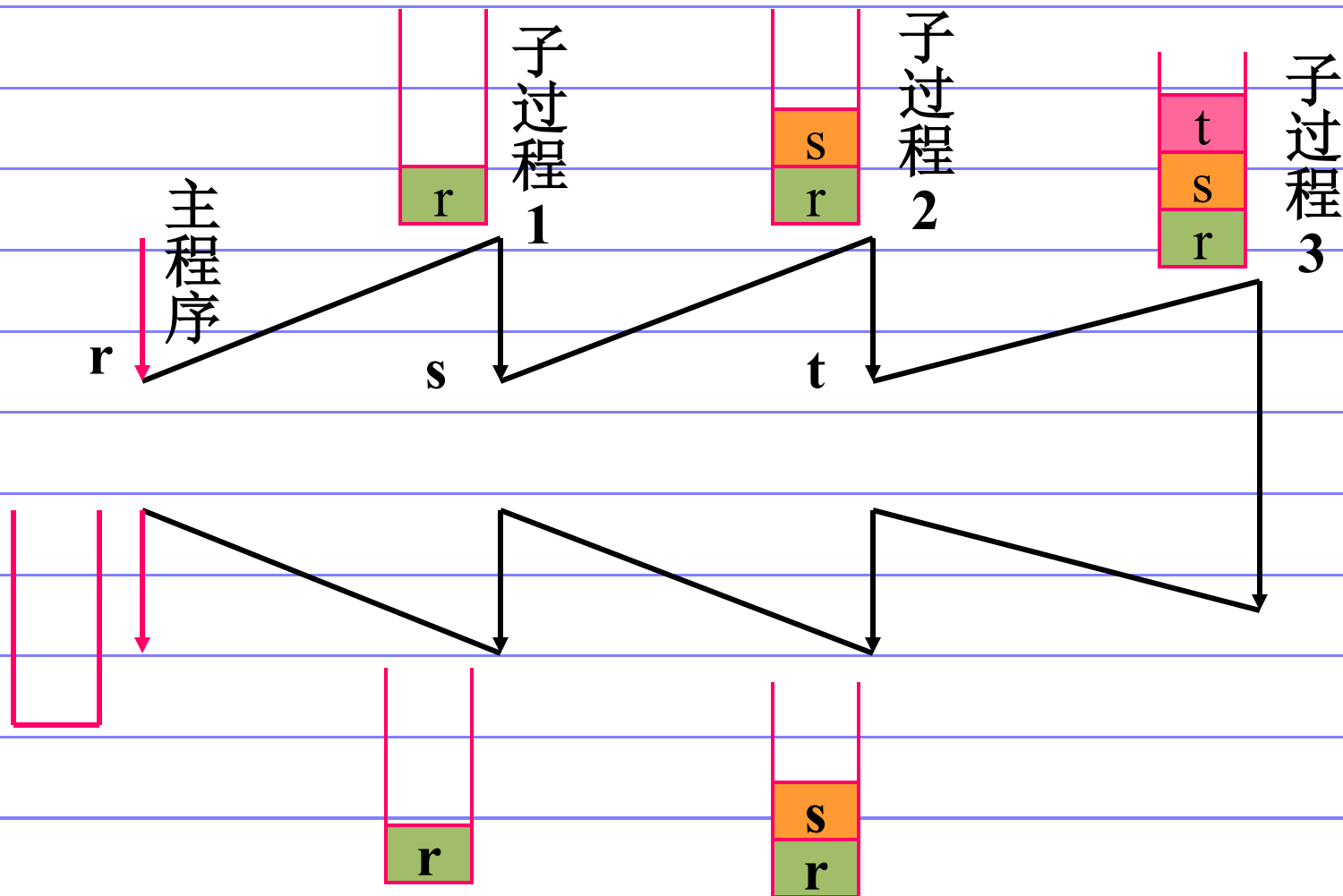
第二节 栈的应用举例

由于栈具有的“后进先出”的固有特性，因此，栈成为程序设计中常用的工具和数据结构。以下是几个栈应用的例子。

- ☐ 过程的嵌套调用
- ☐ 数制转换
- ☐ 括号的匹配
- ☐ 回文游戏
- ☐ 行编辑程序
- ☐ 表达式求值

过程的嵌套调用

□ 子程序调用-调用时入栈，返回时出栈



栈与递归调用的实现

栈的另一个重要应用是在程序设计语言中实现递归调用。

递归调用：一个函数(或过程)直接或间接地调用自己本身，简称**递归**(Recursive)。

递归是程序设计中的一个强有力的工具。因为递归函数结构清晰，程序易读，正确性很容易得到证明。

为了使递归调用不至于无终止地进行下去，实际上有效的递归调用函数(或过程)应包括两部分：

- (1) 递推规则(方法)
- (2) 终止条件。

例如：求 $n!$

栈与递归调用的实现

为保证递归调用正确执行，系统设立一个“递归工作栈”，作为整个递归调用过程期间使用的数据存储区。

每一层递归包含的信息如：参数、局部变量、上一层的返回地址构成一个“工作记录”。每进入一层递归，就产生一个新的工作记录压入栈顶；每退出一层递归，就从栈顶弹出一个工作记录。

$$\text{Fact}(n) = \begin{cases} 1 & \text{当 } n=0 \text{ 时} & \text{终止条件} \\ n * \text{fact}(n-1) & \text{当 } n > 0 \text{ 时} & \text{递推规则} \end{cases}$$

栈与递归调用的实现

从被调函数返回调用函数的一般步骤：

- ① 若栈为空，则执行正常返回。
- ② 从栈顶弹出一个工作记录。
- ③ 将“工作记录”中的参数值、局部变量值赋给相应的变量；读取返回地址。
- ④ 将函数值赋给相应的变量。
- ⑤ 转移到返回地址。

Hanoi塔问题

3 阶 Hanoi 塔假设有三个命名为 A B C 的塔座，在塔座A上插有n个直径大小不相同，由小到大编号为1，2，3，···，n的圆盘，要求将A座上的圆盘移至塔座C并按同样的顺序叠排圆盘移动必须遵守下列规则：1：每次只能移动一个圆盘 2：圆盘可以插在任意一个塔座上 3：任何时刻都不能将一个较大的圆盘放在一个较小的圆盘上

```
#include <stdio.h>

void hanoi(int i, char A, char B, char C);
void move(int i, char x, char y);

int main()
{
    int n;
    printf("请输入n的值: ");
    scanf("%d", &n);
    hanoi(n, 'A', 'B', 'C');
    return 0;
}
```

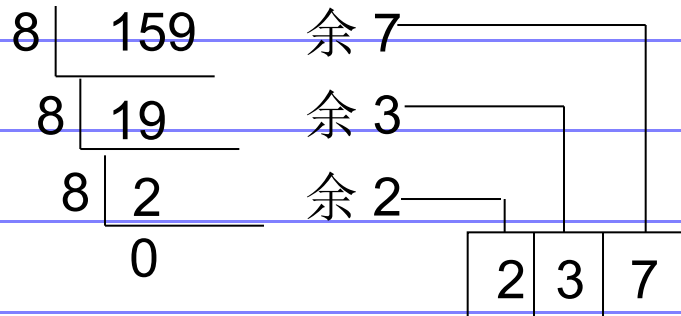
Hanoi塔问题

```
void hanoi(int i , char A , char B , char C)
{
    if(i == 1)
    {
        move(i , A , C);
    }
    else
    {
        hanoi(i - 1 , A , C , B); //函数递归调用
        move(i , A , C);
        hanoi(i - 1 , B , A , C);
    }
}

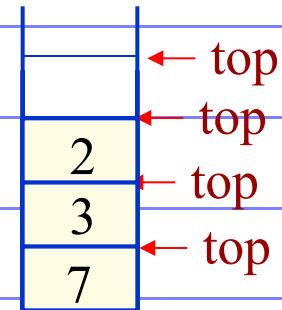
void move(int i , char x , char y)
{
    static int c = 1; //局部变量i声明为 static
    printf("%d: %d from %c ——> %c %n", c++ , i , x , y);
}
```

数制转换

□ 把十进制数159转换成八进制数



$$(159)_{10} = (237)_8$$



数制转换

采用静态顺序栈方式实现

```
void conversion(int n , int d)
```

```
/*将十进制整数N转换为d(2或8)进制数*/
```

```
{ SqStack S ;   int k, *e ;
```

```
  S=Init_Stack();
```

```
  while (n>0) { k=n%d ; push(S , k) ; n=n/d ;   }
```

```
    /* 求出所有的余数，进栈 */
```

```
  while (S.top!=-1)    /* 栈不空时出栈，输出 */
```

```
    { pop(S, e) ;
```

```
      printf("%1d" , *e) ;
```

```
    }
```

```
}
```

括号的匹配

□ 考虑下列括号序列

■ `[([[]])]`

■ 要求成对出现,但可以嵌套

□ 进一步考虑算术表达式中的括号,只允许大(花)括号中套中括号、中括号中套小括号

■ `{[()]}`

□ 考虑字符串,在JavaScript脚本语言中,字符串是可以用双引号” ”或单引号’ ’包括的一个串。用双引号括起的串中可以有单引号,而用单引号括起的串中可以有双引号。

括号的匹配

在文字处理软件或编译程序设计中，常常需要检查一个字符串或一个表达式中的括号是否相匹配？

匹配思想：从左至右扫描一个字符串(或表达式)，则每个右括号将与最近遇到的那个左括号相匹配。则可以在从左至右扫描过程中把所遇到的左括号存放到堆栈中。每当遇到一个右括号时，就将它与栈顶的左括号(如果存在)相匹配，同时从栈顶删除该左括号。

算法思想：设置一个栈，当读到左括号时，左括号进栈。当读到右括号时，则从栈中弹出一个元素，与读到的左括号进行匹配，若匹配成功，继续读入；否则匹配失败，返回**FLASE**。

括号的匹配

```
#define TRUE 0
#define FLASE -1
SqStack S;
S=Init_Stack(); /*堆栈初始化*/
int Match_Brackets( )
{
    char ch , x ;
    scanf("%c" , &ch) ;
    while (asc(ch)!=13)
    {
        if ((ch=='(')||(ch=='[')) push(S , ch) ;
        else if (ch==']')
        {
            x=pop(S) ;
            if (x!='(')
            {
                printf("[ ' 括号不匹配" ) ;
                return FLASE ;
            }
        }
        else if (ch==')')
        {
            x=pop(S) ;
            if (x!='(')
            {
                printf("( ' 括号不匹配" ) ;
                return FLASE ;
            }
        }
    }
    if (S.top!=-1)
    {
        printf("括号数量不匹配！" ) ;
        return FLASE ;
    }
    else return TRUE ;
}
```

回文游戏

□ 顺读与逆读字符串一样(不含空格)

□ 例如：

■ ini

■ madam im adam

□ 算法

■ 读入字符串

■ 去掉空格（原串）

■ 压入栈

■ 原串字符与出栈字符依次比较

◇ 若不等，非回文

◇ 若直到栈空都相等，回文

行编辑程序

```
//输入数据存入缓冲区用来接收用户输入的一行字符, 之后逐行存入用户数据区
//当用户输入出错时可以输入退格符 # 来表示前一个字符无效
//输入@ 表示当前行中之前输入的字符无效, 读取一行直接打印, 不存入数据区
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#define STACK_INIT_SIZE 100
#define STACKINCREMENT 10
#define OK 1
#define ERROR 0
#define OVERFLOW -2
typedef char SElemtype;
typedef int Status;
typedef struct
{
    SElemtype *top;
    SElemtype *base;
    int stacksize;
} SqStack;
```

行编辑程序

```
Status InitStack(SqStack *s)
```

```
{
```

```
    s->base=(SElemtype*)malloc(STACK_INIT_SIZE*sizeof(SElemtype));
```

```
    if (!s->base)    exit(OVERFLOW);
```

```
    s->top = s->base;
```

```
    s->stacksize = STACK_INIT_SIZE;
```

```
    return OK;
```

```
}
```

```
Status Push(SqStack *s, SElemtype c)
```

```
{
```

```
    if (s->top - s->base == s->stacksize)    {
```

```
        s->base = (SElemtype *)realloc(s->base, (s->stacksize + STACKINCREAMENT) *  
sizeof(SElemtype));
```

```
        if (!s->base)    exit(OVERFLOW);
```

```
        s->top = s->base + s->stacksize;
```

```
        s->stacksize += STACKINCREAMENT;
```

```
    }
```

```
    s->top++;
```

```
    *(s->top) = c;
```

```
    return OK;
```

```
}
```

行编辑程序

```
Status Pop(SqStack *s)
```

```
{
```

```
    if (s->top == s->base)        return ERROR;
```

```
    --s->top; //此处弹栈不需要返回栈顶元素，直接删除即可
```

```
}
```

```
Status PrintStack(SqStack *s)
```

```
{
```

```
    int i = 0;
```

```
    while((s->base + i) != (s->top))//遍历栈，从栈底+1开始直到栈顶为止
```

```
    {
```

```
        i++;
```

```
        printf("%c", *(s->base + i));
```

```
    }
```

```
    return OK;
```

```
}
```

```
Status ClearStack(SqStack *s)
```

```
{
```

```
    s->top = s->base;
```

```
    return OK;
```

```
}
```

行编辑程序

```
/* 通过getchar()接收当前输入的一个字符 */
```

```
/* 输入为 非#@ 时压栈 */
```

```
/* 输入为 # 时弹栈删除上一个字符 */
```

```
/* 输入为 @ 时清空栈中元素 */
```

```
/* 输入为 \n 时清栈 */
```

```
/* 输入为 EOF 时结束输入 */
```

```
void LineEdit(SqStack *s, SElemtype c)
```

```
{
```

```
    switch(c) {
```

```
        case '#':
```

```
            Pop(s);
```

```
            break;
```

```
        case '@':
```

```
            ClearStack(s);
```

```
            break;
```

```
        default:
```

```
            Push(s, c);
```

```
            break;
```

```
    }
```

```
}
```

行编辑程序

```
int main(void)
{
    SqStack s;
    SElemtype c, *temp;
    InitStack(&s);
    c = getchar();
    while(c != EOF)
    {
        while(c != EOF && c != '\n')
        {
            LineEdit(&s, c);
            c = getchar();
        }
        PrintStack(&s);
        ClearStack(&s);
        printf("\n");
        if(c != EOF)    c = getchar(); // 读取下一行的第一个字符
    }
    return 0;
}
```

表达式求值

□ 运算规则：

- 从左到右
- 先乘除、后加减
- 先括号内、后括号外

□ 中缀表达式

$a*b+c$

$a+b*c$

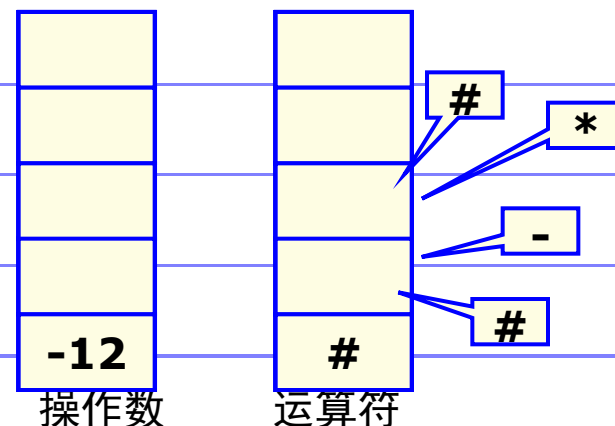
$a+(b*c+d)/e$

□ 操作数栈和运算符栈

□ 例 计算 $2+4-3*6$

算符间的优先关系

$\theta_1 \backslash \theta_2$	+	-	*	/	(	)	#
+	>	>	<	<	<	>	>
-	>	>	<	<	<	>	>
*	>	>	>	>	<	>	>
/	>	>	>	>	<	>	>
(	<	<	<	<	<	=	
)	>	>	>	>		>	>
#	<	<	<	<	<		=



表达式求值

为了通过代码实现，我们定义两个工作栈，一个叫OPTR，存运算符和界限符；另一个存OPND，存操作数或运算结果。首先OPND为空栈，OPTR首先存‘#’为栈底元素。依次读取算术表达式，如果是操作数，存OPND；如果是运算符或界限符，则和OPTR的栈顶元素比较优先级，如果优先级高，存入栈，如果优先级低，则进行运算操作。

表达式求值

```
#include<stdio.h>
#include<stdlib.h>
#define STACK_INIT_SIZE 100//存储空间初始分配量
#define STACKINCREMENT 10//分配增量
typedef int Status;
typedef char ElemType;
typedef struct{
    ElemType *base;
    ElemType *top;
    int stacksize;
}sqStack;
Status InitStack(sqStack &s){
    s.base=(ElemType *)malloc(STACK_INIT_SIZE*sizeof(ElemType));
    if(!s.base){
        exit(1);
    }
    s.top=s.base;
    s.stacksize=STACK_INIT_SIZE;
    return 0;
}
```

表达式求值

```
ElemType GetTop(sqStack s){//获取栈顶元素
```

```
    ElemType e;
```

```
    if(s.top==s.base) return 0;
```

```
    e=*(s.top-1);
```

```
    return e;
```

```
}
```

```
Status Push(sqStack &s,ElemType e){//插入元素e为新的栈顶元素
```

```
    if(s.top-s.base>=s.stacksize){//如果栈满，扩充空间
```

```
        s.base = (ElemType
```

```
*)realloc(s.base,(s.stacksize+STACKINCREMENT)*sizeof(ElemType));
```

```
        if(!s.base) exit(1);
```

```
        s.top=s.base+s.stacksize;
```

```
        s.stacksize+=STACKINCREMENT;
```

```
    }
```

```
    *s.top++=e;//赋值后栈顶指针+1
```

```
    return 0;
```

```
}
```

表达式求值

```
Status Pop(sqStack &s, ElemType &e){ //删除栈顶元素
    if(s.top == s.base) return 1;
    e = *--s.top; //栈顶指针-1, 给e赋值
    return 0;
}
```

```
Status In(ElemType c){
    if(c == '+' || c == '-'
        || c == '*' || c == '/' || c == '#' || c == '(' || c == ')' || c == '[' || c == ']')
        return 1;
    else
        return 0;
}
```

表达式求值

char Precede(ElemType a, ElemType b){//判断运算符栈的 栈顶元素a和读入元素b的优先级

```
    if(a=='+'||a=='-') {  
        if(b=='+'||b=='-'||b=='>'||b=='#'||b=='') return '>';  
        else return '<';  
    }
```

```
    if(a=='*'||a=='/') {  
        if(b=='('||b=='[') return '<';  
        else return '>';  
    }
```

```
    if(a=='(') {  
        if(b=='') return '=';  
        else return '<';  
    }
```

```
    if(a=='[') {  
        if(b==']') return '=';  
        else return '<';  
    }
```

```
    if(a=='#') {  
        if(b=='#') return '=';  
        else return '<';  
    }
```

```
    }  
} //Precede
```

表达式求值

```
ElemType Operate(ElemType a, ElemType x, ElemType b){  
    //进行运算的函数  
    switch (x){  
        case '+':  
            return a+b;  
        case '-':  
            return a-b;  
        case '*':  
            return a*b;  
        case '/':  
            return a/b;  
    }  
} //Operator
```

表达式求值

```
ElemType EvaluateExpression(){
    sqStack OPTR,OPND;//OPTR:运算符栈 OPND:运算数栈
    char c, x;
    InitStack(OPTR); Push(OPTR,'#'); InitStack(OPND); c=getchar();
    while(c!='#' || GetTop(OPTR)!='#'){
        if(!In(c)){
            Push(OPND,c); //不是运算符, 进入运算数栈。
            c=getchar();
        }
        else
            switch(Precede(GetTop(OPTR),c)){
                case '<':Push(OPTR,c);c=getchar();break;//栈顶元素优先级低
                case '=':Pop(OPTR,x); c=getchar();break;//栈顶元素优先级低
                case '>':
                    Pop(OPTR,x);
                    ElemType a, b;
                    Pop(OPND, b);
                    Pop(OPND, a);
                    Push(OPND, Operate(a, x, b));
                    break;
            }
    }
    return GetTop(OPND);
}
```

第三节 队列

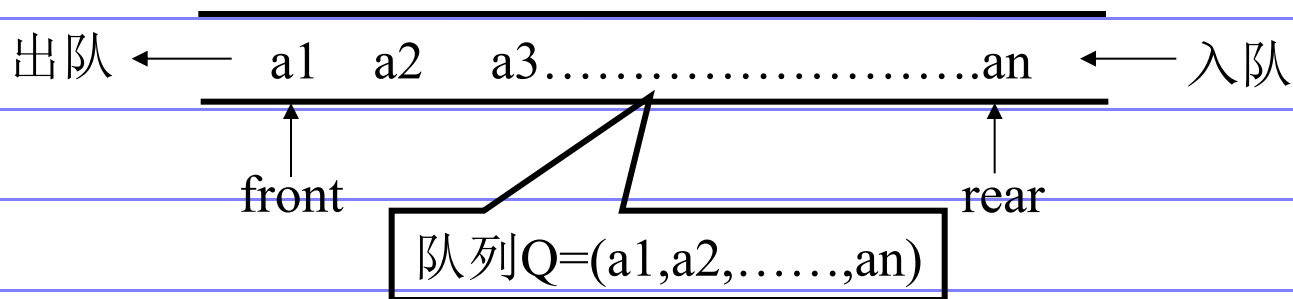
□ 队列的定义及特点

■ 定义：队列是限定只能在表的一端进行插入，在表的另一端进行删除的线性表（运算受限的线性表）

◇ 队尾(rear)——允许插入的一端

◇ 队头(front)——允许删除的一端

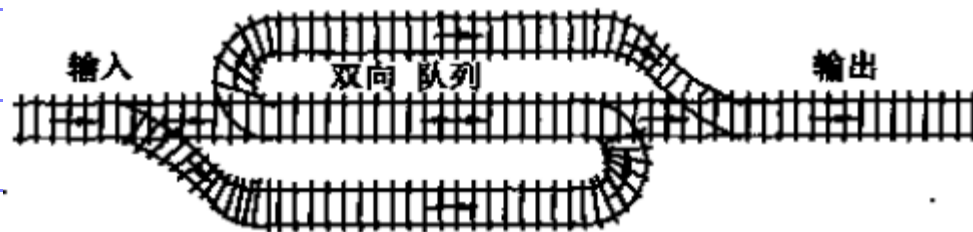
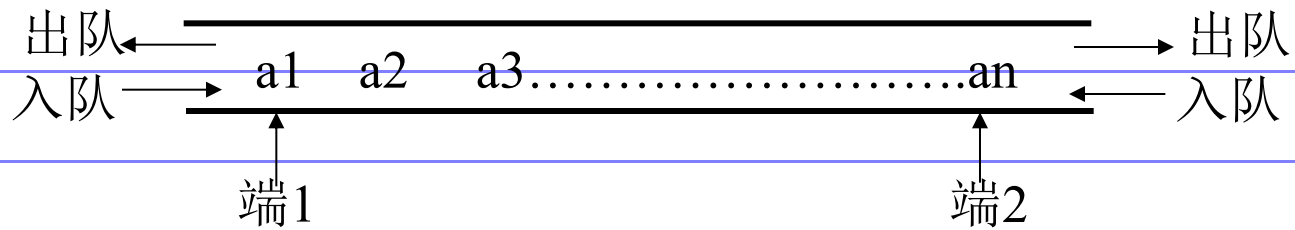
■ 队列特点：先进先出(FIFO)



例如：排队购物。操作系统中的作业排队。先进入队列的成员总是先离开队列。

双端队列

□ 定义：限定插入和删除操作可在表的两端进行的线性表。



队列的抽象描述

□ ADT Queue{

数据对象: $D = \{ a_i | a_i \in \text{ElemSet}, i=1, 2, \dots, n, n \geq 0 \}$

◇数据关系: $R = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2,3,\dots,n \}$

约定 a_1 端为队首, a_n 端为队尾。

◇基本操作:

◇**Create()**: 创建一个空队列;

◇**EmptyQue()**: 若队列为空, 则返回**true**, 否则返回**false**;

◇.....

◇**InsertQue(x)**: 向队尾插入元素 x ;

◇**DeleteQue(x)**: 删除队首元素 x ;

■ } ADT Queue

队列的抽象描述

ADT Queue {

数据对象: $D = \{ a_i \mid a_i \in \text{ElemSet}, i = 1, 2, \dots, n, n \geq 0 \}$

数据关系: $R1 = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i = 2, \dots, n \}$
约定其中 a_1 端为队列头,
 a_n 端为队列尾。

基本操作:

InitQueue(&Q)

操作结果:构造一个空队列 Q。

DestroyQueue(&Q)

初始条件:队列 Q 已存在。

操作结果:队列 Q 被销毁,不再存在。

ClearQueue(&Q)

初始条件:队列 Q 已存在。

操作结果:将 Q 清为空队列。

QueueEmpty(Q)

初始条件:队列 Q 已存在。

操作结果:若 Q 为空队列,
则返回 TRUE, 否则 FALSE。 | ADT Queue

QueueLength(Q)

初始条件:队列 Q 已存在。

操作结果:返回 Q 的元素个数,
即队列的长度。

GetHead(Q, &e)

初始条件:Q 为非空队列。

操作结果:用 e 返回 Q 的队头元素。

EnQueue(&Q, e)

初始条件:队列 Q 已存在。

操作结果:插入元素 e 为 Q 的新的
队尾元素。

DeQueue(&Q, &e)

初始条件:Q 为非空队列。

操作结果:删除 Q 的队头元素,
并用 e 返回其值。

QueueTraverse(Q, visit())

初始条件:Q 已存在且非空。

操作结果:从队头到队尾,依次对 Q 的
每个数据元素调用函数 visit(),
一旦 visit() 失败,则操作失败。

队列的顺序表示

利用一组连续的存储单元(一维数组)依次存放从队首到队尾的各个元素,称为顺序队列。

对于队列,和顺序栈相类似,也有动态和静态之分。本部分介绍的是静态顺序队列,其类型定义如下:

```
❑ #define MAX_QUEUE_SIZE 100
```

```
❑ typedef struct queue
```

```
    ■ { ElemType Queue_array[MAX_QUEUE_SIZE];
```

```
        ✧ int front;
```

```
        ✧ int rear;
```

```
    ■ }SqQueue;
```

队列的顺序表示

□ 设立一个队首指针`front`，一个队尾指针`rear`，分别指向队首和队尾元素。

◆ 初始化：`front=rear=0`。

◆ 入队：将新元素插入`rear`所指的位置，然后`rear`加1。

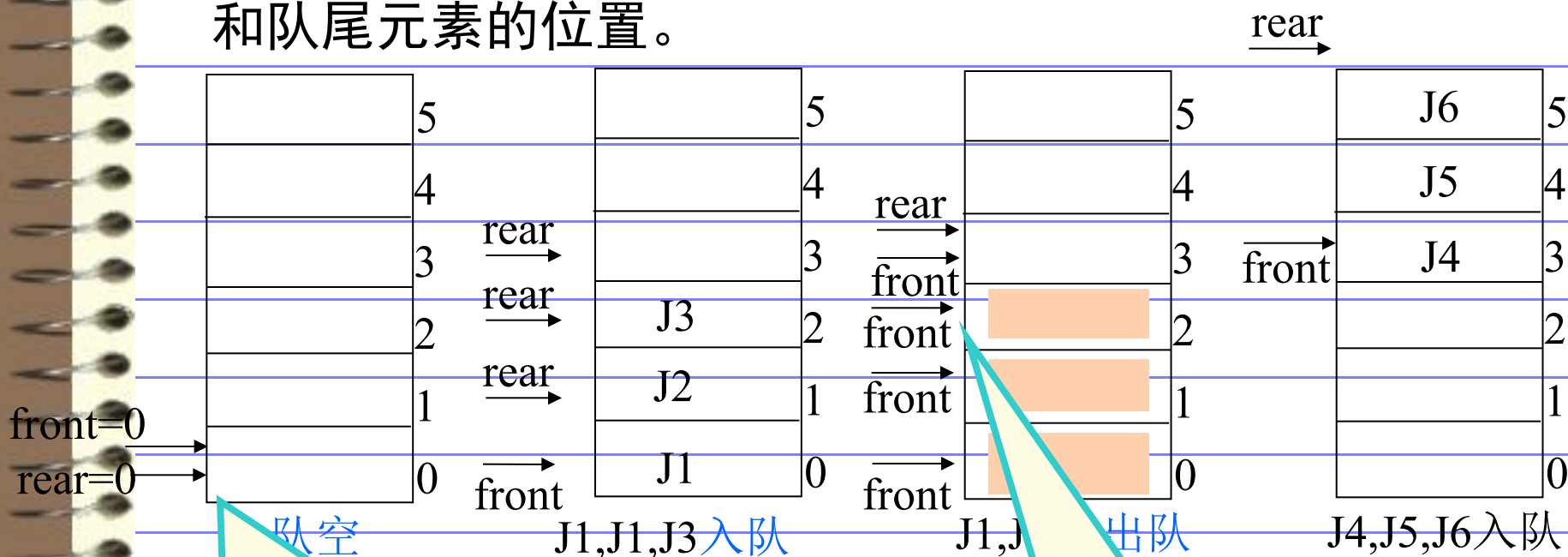
◆ 出队：删去`front`所指的元素，然后加1并返回被删元素。

◆ 队列为空：`front=rear`。

◆ 队满：`rear=MAX_QUEUE_SIZE`

队列的顺序表示

❑ 队列作为线性表，也可以用顺序存储结构存储。除了可以用一组地址连续的存储单元依次存放从队头到队尾的元素之外，还需附设两个指针front和rear分别指向队头元素和队尾元素的位置。



设两个指针front, rear, 约定:
rear指示队尾元素的下一个位置;
front指示队头元素
初值front=rear=0

空队列条件: $\text{front} == \text{rear}$
入队列: $\text{sq}[\text{rear}++] = \text{x};$
出队列: $\text{x} = \text{sq}[\text{front}++];$

顺序队列存在的问题

□ 在非空队列里，队首指针始终指向队头元素，而队尾指针始终指向队尾元素的下一位置。

□ 设数组维数为M，则-

■ 当 $\text{front}=0, \text{rear}=M$ 时，当再次有元素入队时发生溢出——真溢出

■ 当 $\text{front} \neq 0, \text{rear}=M$ 时，当再次有元素入队时发生溢出——假溢出

因为在入队和出队操作中，头、尾指针只增加不减小，致使被删除元素的空间永远无法重新利用。因此，尽管队列中实际元素个数可能远远小于数组大小，但可能由于尾指针已超出向量空间的上界而不能做入队操作。

顺序队列存在的问题

□ 解决方案

- 队首固定，每次出队剩余元素向下移动——浪费时间
- 循环队列

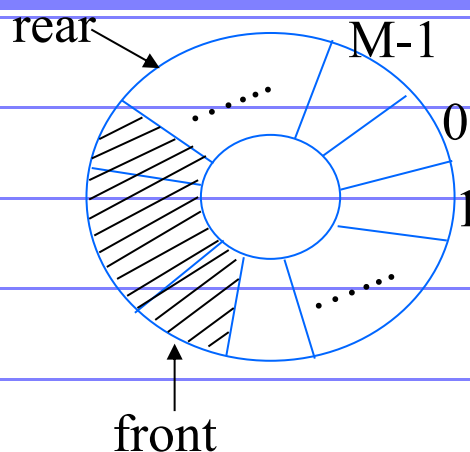
将为队列分配的向量空间看成为一个首尾相接的圆环，并称这种队列为循环队列(**Circular Queue**)。

✧ 基本思想：把队列设想成环形，让sq[0]接在sq[M-1]之后，若rear==M，则令rear=0；

✧ 重复利用已经用过的空间

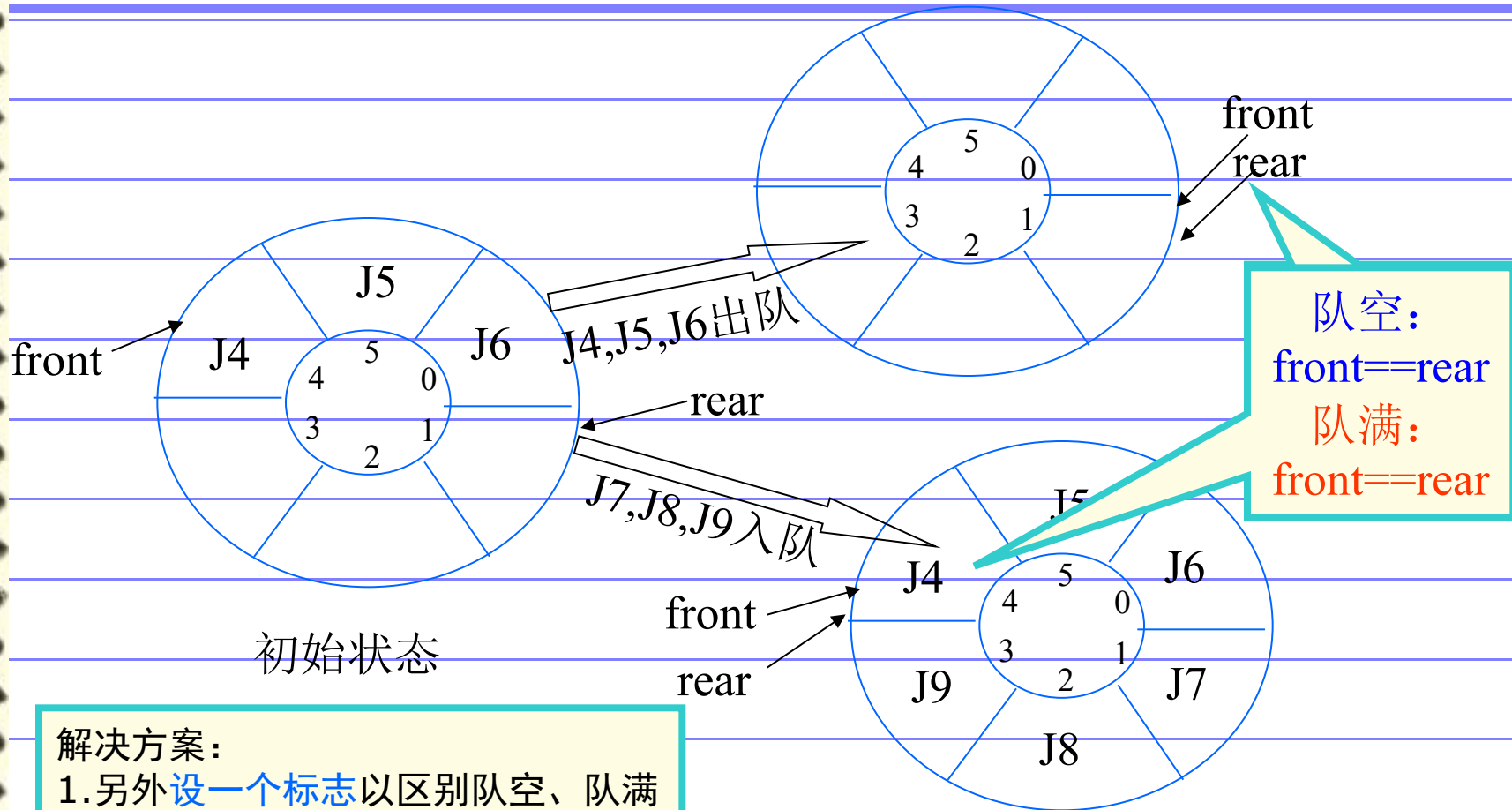
显然，为循环队列所分配的空间可以被充分利用，除非向量空间真的被队列元素全部占用，否则不会上溢。因此，真正实用的顺序队列是循环队列。

循环队列



- 基本思想：把队列sq设想成环形，让sq[0]接在sq[M-1]之后，若 $\text{rear}+1==M$ ，则令 $\text{rear}=0$ ；
- 实现：利用“模”运算
 - 入队： $\text{sq}[\text{rear}]=x$; $\text{rear}=(\text{rear}+1)\%M$;
 - 出队： $x=\text{sq}[\text{front}]$; $\text{front}=(\text{front}+1)\%M$;

循环队列队满、队空判定条件



解决方案:

1. 另外设一个标志以区别队空、队满

2. 少用一个元素空间:

队空: $\text{front} == \text{rear}$

队满: $(\text{rear} + 1) \% M == \text{front}$

循环队列队满、队空判定条件

入队时尾指针向前追赶头指针，出队时头指针向前追赶尾指针，故队空和队满时头尾指针均相等。因此，无法通过 $front=rear$ 来判断队列“空”还是“满”。解决此问题的方法是：约定入队前，测试尾指针在循环意义下加1后是否等于头指针，若相等则认为队满。即：

- ◆ $rear$ 所指的单元始终为空。
- ◆ 循环队列为空： $front=rear$ 。
- ◆ 循环队列满： $(rear+1)\%MAX_QUEUE_SIZE = front$ 。

循环队列的顺序存储

```
#define MAXQSIZE 100
```

```
Typedef struct {
```

```
    QElemType *base;
```

```
    int front;
```

```
    int rear;
```

```
}SqQueue;
```

```
Status InitQueue(SqQueue &Q) {
```

```
    Q.base = (QElemType *)malloc(MAXQSIZE*sizeof(QElemType));
```

```
    if(!Q.base) exit(OVERFLOW);
```

```
    Q.front = Q.rear = 0;
```

```
    return OK;
```

```
}
```

循环队列插入与删除

```
Status EnQueue(SqQueue &Q, QElemType e) {  
    if((Q.rear+1)%MAXSIZE) == Q.front) return ERROR;  
    Q.base[Q.rear] = e;  
    Q.rear = (Q.rear+1)%MAXQSIZE;  
    return OK;  
} //入队
```

```
Status DeQueue(SqQueue &Q, QElemType &e) {  
    if(Q.front == Q.rear) return ERROR;  
    e = Q.base[Q.front];  
    Q.front = (Q.front+1)%MAXQSIZE;  
    return OK;  
} //出队
```

队列的链式表示和实现

队列的链式存储结构简称为链队列，它是限制仅在表头进行删除操作和表尾进行插入操作的单链表。

需要两类不同的结点：**数据元素结点**，队列的**队首指针**和**队尾指针**的结点，如下图所示。

数据元素结点类型定义：

```
typedef struct Qnode
```

```
{ ElemType data ;
```

```
    struct Qnode *next ;
```

```
}QNode ;
```

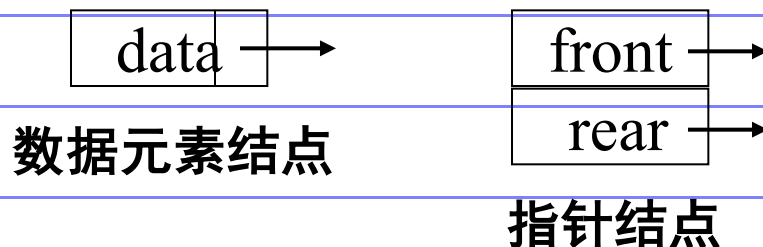


图 链队列结点示意图

队列的链式表示和实现

队列的链式存储结构简称为链队列，它是限制仅在表头进行删除操作和表尾进行插入操作的单链表。

需要两类不同的结点：**数据元素结点**，队列的**队首指针**和**队尾指针**的结点，如下图所示。

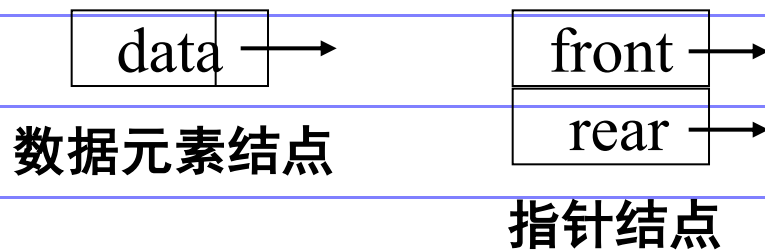


图 链队列结点示意图

队列的链式表示和实现

数据元素结点类型定义：

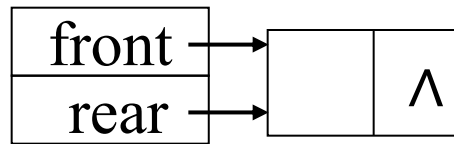
```
typedef struct Qnode
{ ElemType data ;
  struct Qnode *next ;
}QNode ;
```

指针结点类型定义：

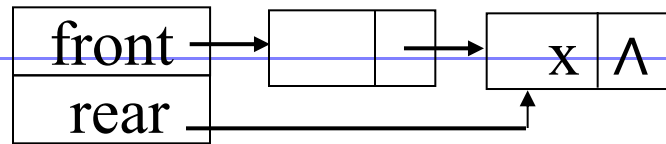
```
typedef struct link_queue
{ QNode *front, *rear ;
}Link_Queue ;
```

链队运算及指针变化

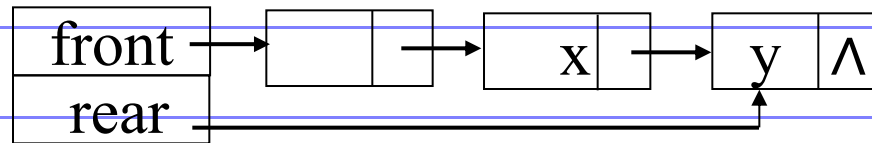
链队的操作实际上是单链表的操作，只不过是删除在表头进行，插入在表尾进行。插入、删除时分别修改不同的指针。链队运算及指针变化如下图所示。



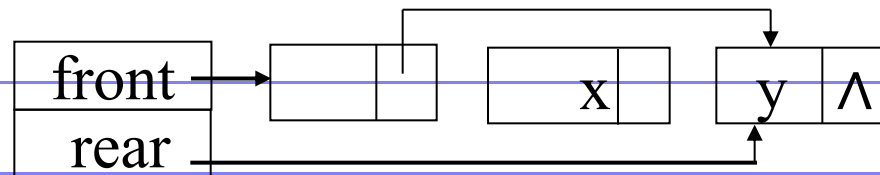
(a) 空队列



(b) x 入队



(c) y 再入队



(d) x 出队

图 队列操作及指针变化

链队列基本操作

链队列的初始化

```
LinkQueue *Init_LinkQueue(void)
{
    LinkQueue *Q ; QNode *p ;
    p=(QNode *)malloc(sizeof(QNode)) ; /* 开辟头结点 */
    p->next=NULL ;
    Q=(LinkQueue *)malloc(sizeof(LinkQueue)) ;
    /* 开辟链队的指针结点 */
    Q.front=Q.rear=p ;
    return(Q) ;
}
```

链队列基本操作

链队列的入队操作

在已知队列的队尾插入一个元素e，即修改队尾指针(Q.rear)。

```
Status Insert_CirQueue(LinkQueue *Q, ElemType e)
```

```
/* 将数据元素e插入到链队列Q的队尾 */
```

```
{ p=(QNode *)malloc(sizeof(QNode));
```

```
if (!p) return ERROR;
```

```
/* 申请新结点失败，返回错误标志 */
```

```
p->data=e ; p->next=NULL ; /* 形成新结点 */
```

```
Q.rear->next=p ; Q.rear=p ; /* 新结点插入到队尾 */
```

```
return OK;
```

```
}
```

链队列基本操作

链队列的出队操作

Status Delete_LinkQueue(LinkQueue *Q, ElemType *x)

```
{ QNode *p ;  
    if (Q.front==Q.rear) return ERROR ; /* 队空 */  
    p=Q.front->next ; /* 取队首结点 */  
    *x=p->data ;  
    Q.front->next=p->next ; /* 修改队首指针 */  
    if (p==Q.rear) Q.rear=Q.front ;  
        /* 当队列只有一个结点时应防止丢失队尾指针 */  
    free(p) ;  
    return OK ;  
}
```

链队列基本操作

链队列的撤消

```
void Destroy_LinkQueue(LinkQueue *Q )  
    /* 将链队列Q的队首元素出队 */  
    { while (Q.front!=NULL)  
        { Q.rear=Q.front->next;  
            /* 令尾指针指向队列的第一个结点 */  
            free(Q.front); /* 每次释放一个结点 */  
            /* 第一次是头结点，以后是元素结点 */  
            Q.front=Q.rear;  
        }  
    }
```

习题二 错题解析

2.29 ⑤ 已知A, B和C为三个递增有序的线性表, 现要求对A表作如下操作: 删去那些既在B表中出现, 又在C表中出现的元素。试对顺序表编写实现上述操作的算法, 并分析你的算法的时间复杂度 (注意: 同一表中各元素值可能相同)。

```
void DeleteShareItem(Node *a, Node *b, Node *c)
{
    Node *p=a->next,*q=b->next,*s=c->next;
    Node *temp=NULL;
    while(q!=NULL) {
        while(s!=NULL) {
            if(q->point==s->point) {
                while(p!=NULL) {
                    if(p->next->point==q->point) {
                        temp=p->next;
                        p->next=temp->next;
                        free(temp);
                        temp=NULL;
                    }
                    else
                        p=p->next;
                }
                break;
            }
            s=s->next;
        }
        s=c->next;
        q=q->next;
    }
}
```

```
SqList *SqList_del_A_in_B_and_C(SqList *A, SqList B, SqList C)
{
    for (int i = 0; i < A->length; i++)
    {
        bool ans_B = Find_in_SqList(B, A->elem[i]);
        bool ans_C = Find_in_SqList(C, A->elem[i]);
        bool ans = ans_B ^ ans_C;
        if (ans == TRUE)
        {
            Del_in_SqList(A, i);
        }
    }
    return A;
}
```

```

void Algo_2_29(SqlList *La, SqlList Lb, SqlList Lc)
{
    int i, j, k, count;

    i = j = k = 0;
    count = 0;

    while(i < (*La).length && j < Lb.length && k < Lc.length)
    {
        if(Lb.elem[j] < Lc.elem[k])
            j++;
        else if(Lb.elem[j] > Lc.elem[k])
            k++;
        else
        {
            while(i < (*La).length && (*La).elem[i] < Lb.elem[j])
            {
                (*La).elem[count] = (*La).elem[i];
                count++;
                i++;
            }

            while(i < (*La).length && (*La).elem[i] == Lb.elem[j])
                i++;

            if(i < (*La).length)
            {
                j++;
                k++;
            }
        }
    }

    while(i < (*La).length)
    {
        (*La).elem[count] = (*La).elem[i];
        count++;
        i++;
    }

    (*La).length = count;
}

```

$$A = A - B \cap C$$

习题三

- 1 设有一个栈，元素进栈的次序为a, b, c。问经过栈操作后可以得到哪些输出序列？
- 2 循环队列的优点是什么？如何判断它的空和满？
- 3 设有一个静态顺序队列，向量大小为MAX，判断队列为空的条件是什么？队列满的条件是什么？
- 4 设有一个静态循环队列，向量大小为MAX，判断队列为空的条件是什么？队列满的条件是什么？

5 一个双向栈S是在同一向量空间内实现的两个栈，它们的栈底分别设在向量空间的两端。试为此双向栈设计初始化InitStack(S)，入栈Push(S,i,x)，出栈Pop(S,i,x)算法，其中i为0或1，用以表示栈号。

6 设Q[0,6]是一个静态顺序队列，初始状态为front=rear=0，请画出做完下列操作后队列的头尾指针的状态变化情况，若不能入对，请指出其元素，并说明理由。

a, b, c, d入队

a, b, c出队

i, j, k, l, m入队

d, i出队

n, o, p, q, r入队

7 假设 $Q[0,5]$ 是一个循环队列，初始状态为 $front=rear=0$ ，请画出做完下列操作后队列的头尾指针的状态变化情况，若不能入队，请指出其元素，并说明理由。

d, e, b, g, h入队

d, e出队

i, j, k, l, m入队

b出队

n, o, p, q, r入队

数据结构题集（C语言版）：3.3, 3.4, 3.13, 3.16, 3.25, 3.31

1 设有一个栈，元素进栈的次序为a, b, c。问经过栈操作后可以得到哪些输出序列？

abc,acb,bac,bca,cba

部分同学考虑顺序不变，答案为以cba的顺序输出1-3个元素

7 假设Q[0,5]是一个循环队列，初始状态为front=rear=0，请画出做完下列操作后队列的头尾指针的状态变化情况，若不能入对，请指出其元素，并说明理由。

d, e, b, g, h入队 d, e出队 i, j, k, l, m入队 b出队 n, o, p, q, r入队



部分同学忘记了循环数组不能全满，将k也入了队列

3.25●试写出求递归函数F(n)的递归算法，并消除递归：
$$F(n) = \begin{cases} n+1 & n=0 \\ n \cdot F(n/2) & n>0 \end{cases}$$

```
/*  
| 题3.25: 递归计算F(n) |  
*/
```

```
/* 方法1: 递归法求解, 优点是算法简单 */
```

```
int Algo_3_25_1(int n)  
{  
    if(n<0)  
        exit(ERROR);  
  
    if(!n)  
        return n + 1;  
    else  
        return n * Algo_3_25_1(n/2);  
}
```

```
/*  
| 题3.25: 迭代计算F(n) |  
*/
```

```
/* 方法2: 消除递归的迭代法, 优点是存储了上一层计算数值 */
```

```
int Algo_3_25_2(int n)  
{  
    int a[n+1];  
    int i;  
  
    if(n<0)  
        exit(ERROR);  
  
    for(i=1, a[0]=1; i<=n; i++)  
        a[i] = i * a[i/2];  
  
    return a[n];  
}
```